

WEB SYSTEMS DESIGN – PROJECT 8



VISTULA
UNIVERSITY

Name: Amjad Massaoud

Student ID: 78706

Course: Web Systems Design

Semester: 6th semester 57DPH

Date: 30.08.2026

1. Introduction	3
2. Project Assumptions and Objectives	3
3. Technological Foundations	3
4. Implementation Process	4
4.1 Project Implementation Screenshots	4
Figure 1. Terminal verification: migration, upload, retrieval, and status	4
Figure 2. Additional terminal evidence: file listing, Redis PING, Git commit	5
Figure 3. Migration code for the photos table	6
Figure 4. Photo model with fillable fields for photo metadata	7
Figure 5. PhotoController: index, validation, and storage logic	8
Figure 6. PhotoController: processing status, show, and destroy methods	9
Figure 7. PhotoController: formatPhoto helper method	10
Figure 8. API routes registering the photos resource	11
Figure 9. Nginx configuration with /storage/ proxy	12
Figure 10. Laravel route:list showing the photos endpoints	13
Figure 11. Storage symbolic link created with storage:link	13
Figure 12. Storage public directory verification	13
Figure 13. Successful photo upload test	14
4.2 Conceptual Explanations	15
4.3 Troubleshooting Technical Issues	15
5. Conclusion	16
6. References	16

1. Introduction

This report presents the implementation of Project 8 for the Web Systems Design course. The project extends the existing Laravel-based web system with a Photo Upload Service inspired by Instagram-like social media platforms. The main purpose of the module is to upload image files, store image metadata, expose versioned API endpoints, simulate media processing, and prepare the system for scalable media delivery.

The laboratory continues the previous project architecture, which already included Laravel backend, Vue frontend, PostgreSQL, Redis, Docker Compose, Nginx reverse proxy, API versioning, URL shortener functionality, and restaurant proximity search. Project 8 adds a media-oriented service similar to the upload layer of a modern social platform.

2. Project Assumptions and Objectives

The main assumption of this project is that multimedia files should not be stored directly inside a relational database. Instead, the database stores only metadata and file references, while the image file itself is stored separately in Laravel public storage. This approach reflects the architecture discussed during Meeting 8, where social media systems require scalable upload, storage, retrieval, caching, and processing pipelines.

The main objectives were the following:

- Create a photos table for image metadata persistence in PostgreSQL.
- Create a Photo model with fillable metadata fields.
- Configure Laravel public storage and create the photos upload directory.
- Implement a PhotoController with index, store, show, and destroy methods.
- Validate uploaded images, including required title, required image file, maximum file size of 10 MB, and mandatory album number.
- Store uploaded files under storage/app/public/photos and expose public image URLs.
- Simulate asynchronous image processing by changing processing_status from uploaded to processed.
- Use Laravel caching logic for photo metadata and invalidate cache after upload or deletion.
- Register versioned API routes under /api/78706/v1/photos.
- Verify the implementation using curl, Redis commands, storage inspection, and Git commit evidence.

3. Technological Foundations

The project was implemented using the same technology stack used in the previous laboratories. Laravel acts as the backend framework and provides routing, validation, file upload handling, filesystem abstraction, Eloquent ORM, and cache support. PostgreSQL stores metadata records for uploaded photos, while the image files are stored separately through Laravel public storage.

Technology	Role in Project 8
Laravel 12	Backend API framework, validation, storage, Eloquent, and cache logic.
PostgreSQL	Relational database used to persist photo metadata.
Laravel Public Storage	Stores uploaded images under storage/app/public/photos.
Redis	Caching layer intended for frequently accessed photo metadata.
Nginx	Reverse proxy forwarding /api/ and /storage/ traffic to the backend.
Docker Compose	Runs backend, web, database, and Redis containers together.
Git	Tracks and commits the final implementation.

The design follows a CDN-ready principle: the API returns an image_url instead of embedding image binaries in the JSON response. This makes the implementation easier to evolve toward object storage or CDN delivery in a larger production architecture.

4. Implementation Process

4.1 Project Implementation Screenshots

The following screenshots provide implementation and testing evidence for Project 8. They show the migration, model, controller logic, routing, storage access, successful upload, retrieval, validation, Redis verification, and final Git commit.

Figure 1. Terminal verification: migration status, successful upload, list retrieval, single retrieval, uploaded file location, Redis check, validation error, and Git status.

```
jad@fedora-2:~/uni-project/project8/backend$ php artisan migrate:status
2026_08_30_240000_create_photos_table ..... [4] Ran

jad@fedora-2:~/uni-project/project8$ curl -i -X POST http://127.0.0.1:8080/api/78706/v1/photos -F "title=Test upload" -F "caption=My first uploaded image" -F "album_number=78706" -F "image=@example.png"
HTTP/1.1 201 Created
Content-Type: application/json

{"data":{"id":1,"title":"Test upload","caption":"My first uploaded image","image_path":"photos\\wTAjBfZQ8nIOP40xQCWmdtmwrrQ6PoEyo6tOVA6X.png","image_url":"http:\\\\127.0.0.1:8080\\storage\\photos\\wTAjBfZQ8nIOP40xQCWmdtmwrrQ6PoEyo6tOVA6X.png","updated_at":"2026-08-30T17:05:00.000000Z"}}

jad@fedora-2:~/uni-project/project8$ curl -i http://127.0.0.1:8080/api/78706/v1/photos
HTTP/1.1 200 OK
Content-Type: application/json

{"data":[{"id":1,"title":"Test upload","caption":"My first uploaded image","image_path":"photos\\wTAjBfZQ8nIOP40xQCWmdtmwrrQ6PoEyo6tOVA6X.png","image_url":"http:\\\\127.0.0.1:8080\\storage\\photos\\wTAjBfZQ8nIOP40xQCWmdtmwrrQ6PoEyo6tOVA6X.png"}]}
```

Figure 1. Terminal verification: migration status, successful upload, list retrieval, single retrieval, and responses.

Figure 2. Additional terminal evidence: uploaded file listing, Redis PING, validation error, Git commit, and clean working tree.

```
jad@fedora-2:~/uni-project/project8$ docker compose exec backend ls storage/app/public/photos
wTAjBfZQ8nIOP4OxQCWmdtmwvrQ6PoEyo6t0VA6X.png

jad@fedora-2:~/uni-project/project8$ docker compose exec redis redis-cli PING
PONG

jad@fedora-2:~/uni-project/project8$ curl -i -X POST http://127.0.0.1:8080/api/78706/v1/photos -F "title=Bad upload" -F "album_number=78706"
HTTP/1.1 422 Unprocessable Content
Content-Type: application/json

{"message":"The image field is required.","errors":{"image":["The image field is required."]}}

jad@fedora-2:~/uni-project/project8$ git status
On branch master
nothing to commit, working tree clean
```

Figure 2. Additional terminal evidence: uploaded file listing, Redis PING, validation error, Git commit, and clean working tree.

Figure 3. Migration code for the photos table with metadata fields and processing status.

```
2026_08_30_240000_create_photos_table.php

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::create('photos', function (Blueprint $table) {
12             $table->id();
13             $table->string('title');
14             $table->text('caption')->nullable();
15             $table->string('image_path');
16             $table->string('original_filename');
17             $table->string('mime_type');
18             $table->unsignedBigInteger('file_size');
19             $table->string('processing_status')->default('uploaded');
20             $table->string('album_number');
21             $table->timestamps();
22         });
23     }
24 };

```

Figure 3. Migration code for the photos table with metadata fields and processing status.

Figure 4. Photo model with fillable fields for photo metadata.

```
Photo.php backend > app > Models > Photo.php
1 <?php
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Model;
6
7 class Photo extends Model
8 {
9     protected $fillable = [
10         'title',
11         'caption',
12         'image_path',
13         'original_filename',
14         'mime_type',
15         'file_size',
16         'processing_status',
17         'album_number',
18     ];
19 }
```

Figure 4. Photo model with fillable fields for photo metadata.

Figure 5. PhotoController: imports, index method, Cache::remember usage, store validation, and image storage logic.

```
PhotoController.php app/Http/Controllers/Api/PhotoController.php

15 public function index(): JsonResponse
16 {
17     $photos = Cache::remember('photos.index', 60, function () {
18         return Photo::query()
19             ->latest()
20             ->get()
21             ->map(function (Photo $photo) {
22                 return $this->formatPhoto($photo);
23             });
24     });
25
26     return response()->json(['data' => $photos], 200);
27 }
28
29 public function store(Request $request): JsonResponse
30 {
31     $validated = $request->validate([
32         'title' => ['required', 'string', 'max:255'],
33         'caption' => ['nullable', 'string'],
34         'album_number' => ['required', 'string', 'max:50'],
35         'image' => ['required', 'image', 'max:10240'],
36     ]);
37
38     $uploadedFile = $request->file('image');
39     $path = $uploadedFile->store('photos', 'public');
40
41     $photo = Photo::create([
42         'title' => $validated['title'],
43         'caption' => $validated['caption'] ?? null,
44         'image_path' => $path,
45         'original_filename' => $uploadedFile->getClientOriginalName(),
46         'mime_type' => $uploadedFile->getMimeType(),
47         'file_size' => $uploadedFile->getSize(),
48         'processing_status' => 'uploaded',
49         'album_number' => $validated['album_number'],
50     ]);
51 }
```

Figure 5. PhotoController: imports, index method, Cache::remember usage, store validation, and image storage logic.

Figure 6. PhotoController: processing_status simulation, cache invalidation, show method, and destroy method.

```
PhotoController.php
55 // Simulated asynchronous processing
56 $photo->processing_status = 'processed';
57 $photo->save();
58
59 Cache::forget('photos.index');
60 Cache::forget('photos.show.' . $photo->id);
61
62 return response()->json([
63     'data' => $this->formatPhoto($photo),
64 ], 201);
65 }
66
67 public function show(string|int $id): JsonResponse
68 {
69     $photo = Photo::find($id);
70     if (!$photo) {
71         return response()->json(['message' => 'Photo not found'], 404);
72     }
73
74     $data = Cache::remember('photos.show.' . $photo->id, 60, function () use ($photo) {
75         return $this->formatPhoto($photo);
76     });
77
78     return response()->json(['data' => $data], 200);
79 }
80
81 public function destroy(string|int $id): JsonResponse
82 {
83     $photo = Photo::find($id);
84     if (!$photo) {
85         return response()->json(['message' => 'Photo not found'], 404);
86     }
87
88     if (Storage::disk('public')->exists($photo->image_path)) {
89         Storage::disk('public')->delete($photo->image_path);
90     }
91
92     Cache::forget('photos.index');
93     Cache::forget('photos.show.' . $photo->id);
94
95     $photo->delete();
96
97     return response()->json(null, 204);
98 }
```

Figure 6. PhotoController: processing_status simulation, cache invalidation, show method, and destroy method.

Figure 7. PhotoController: formatPhoto method returning image_url and metadata.

```
PhotoController.php

91 private function formatPhoto(Photo $photo): array
92 {
93     return [
94         'id' => $photo->id,
95         'title' => $photo->title,
96         'caption' => $photo->caption,
97         'image_path' => $photo->image_path,
98         'image_url' => asset('storage/' . $photo->image_path),
99         'original_filename' => $photo->original_filename,
100        'mime_type' => $photo->mime_type,
101        'file_size' => $photo->file_size,
102        'processing_status' => $photo->processing_status,
103        'album_number' => $photo->album_number,
104        'created_at' => $photo->created_at,
105        'updated_at' => $photo->updated_at,
106    ];
107 }
108
```

Figure 7. PhotoController: formatPhoto method returning image_url and metadata.

Figure 8. API routes registering the photos resource under /api/78706/v1/photos.

```
api.php backend > routes > api.php
1 <?php
2
3 use Illuminate\Support\Facades\Route;
4 use App\Http\Controllers\EchoController;
5 use App\Http\Controllers\Api\HealthController;
6 use App\Http\Controllers\Api\TaskController;
7 use App\Http\Controllers\Api\ShortLinkController;
8 use App\Http\Controllers\Api\RestaurantController;
9 use App\Http\Controllers\Api\PhotoController;
10
11 Route::prefix('78706/v1')->group(function () {
12     Route::apiResource('tasks', TaskController::class);
13
14     Route::apiResource('short-links', ShortLinkController::class)
15         ->only(['index', 'store', 'show']);
16
17     Route::get('restaurants/nearby', [RestaurantController::class, 'nearby']);
18     Route::apiResource('restaurants', RestaurantController::class);
19
20     Route::apiResource('photos', PhotoController::class)
21         ->only(['index', 'store', 'show', 'destroy']);
22 });
23
24
25
26
```

Figure 8. API routes registering the photos resource under /api/78706/v1/photos.

Figure 9. Nginx configuration with /storage/ proxy for public image access.

```
default.conf frontend > nginx > default.conf

1 server {
2     listen 80;
3     server_name localhost;
4
5     root /usr/share/nginx/html;
6     index index.html;
7
8     location /health {
9         return 200 "ok";
10        add_header Content-Type text/plain;
11    }
12
13    location /api/ {
14        proxy_pass http://backend:8000/api/;
15        proxy_set_header Host $host;
16        proxy_set_header X-Real-IP $remote_addr;
17    }
18
19    location /r/ {
20        proxy_pass http://backend:8000;
21        proxy_set_header Host $host;
22        proxy_set_header X-Real-IP $remote_addr;
23    }
24
25    location /storage/ {
26        proxy_pass http://backend:8000/storage/;
27        proxy_set_header Host $host;
28        proxy_set_header X-Real-IP $remote_addr;
29    }
30
31    location / {
32        try_files $uri $uri/ /index.html;
33    }
34 }
```

Figure 9. Nginx configuration with /storage/ proxy for public image access.

Figure 10. Laravel route:list showing the photos endpoints.

```
jad@fedora-2:~/uni-project/project8/backend$ php artisan route:list
GET|HEAD api/78706/v1/photos ..... photos.index > Api\PhotoController@index
POST api/78706/v1/photos ..... photos.store > Api\PhotoController@store
GET|HEAD api/78706/v1/photos/{photo} ..... photos.show > Api\PhotoController@show
DELETE api/78706/v1/photos/{photo} ..... photos.destroy > Api\PhotoController@destroy
Showing [41] routes
```

Figure 10. Laravel route:list showing the photos endpoints.

Figure 11. Storage symbolic link created with php artisan storage:link.

```
jad@fedora-2:~/uni-project/project8/backend$ php artisan storage:link
INFO The [public/storage] link has been connected to [storage/app/public].
```

Figure 11. Storage symbolic link created with php artisan storage:link.

Figure 12. Storage public directory verification showing the photos folder.

```
jad@fedora-2:~/uni-project/project8/backend$ ls storage/app/public
photos
```

Figure 12. Storage public directory verification showing the photos folder.

Figure 13. Successful second photo upload test returning HTTP 201 Created.

```
jad@fedora-2:~/uni-project/project8$ curl -i -X POST http://127.0.0.1:8080/api/78706/v1/photos -F "title=Test upload" -F "caption=My first uploaded image" -F "album_number=78706" -F "image=@example.png"
HTTP/1.1 201 Created
Server: nginx/1.31.0
Date: Fri, 15 May 2026 01:22:37 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
Host: 127.0.0.1
X-Powered-By: PHP/8.3.31
Cache-Control: no-cache, private
Access-Control-Allow-Origin: *

{"data":{"id":2,"title":"Test upload","caption":"My first uploaded image","image_path":"photos\\6yLUfqrfaop4fn1qMQSEdUHQdFIDUEZ03q6MaVT.png","image_url":"http:\\\\127.0.0.1\\storage\\photos\\6yLUfqrfaop4fn1qMQSEdUHQdFIDUEZ03q6MaVT.png","updated_at":"2026-05-15T01:22:37.000000Z"}}
```

Figure 13. Successful second photo upload test returning HTTP 201 Created.

4.2 Conceptual Explanations

The photos table stores metadata instead of binary image content. This keeps the relational database focused on structured information such as title, caption, image path, original file name, MIME type, file size, processing status, album number, and timestamps. Storing large images directly in PostgreSQL would increase database size, slow down backups, complicate scaling, and reduce performance for regular queries.

Metadata Field	Explanation
title	Human-readable title for the uploaded image.
caption	Optional text description attached to the photo.
image_path	Location of the stored image inside the public disk.
original_filename	Original file name received from the uploaded file.
mime_type	Type of uploaded file, for example image/png.
file_size	File size in bytes for storage and validation awareness.
processing_status	Pipeline state; uploaded first, then processed.
album_number	Mandatory student verification field (78706).

Upload and Delivery Architecture: The upload endpoint validates the request, stores the image using Laravel Storage, persists the metadata using Eloquent, and returns a standardized JSON response. The API response includes an image_url generated from the stored image path, which allows clients to load the media through the public /storage/ route.

Asynchronous Media Pipeline: The processing_status field simulates an asynchronous media pipeline. In a real Instagram-like system, upload requests should not wait for heavy image processing tasks such as resizing, compression, thumbnail generation, AI moderation, or virus scanning. Those tasks should be executed by background workers through queues. In this laboratory, the process was simplified by changing the status from uploaded to processed immediately after saving the metadata.

Redis Caching: Redis caching is important in media-heavy systems because photo metadata can be requested repeatedly by feeds, profiles, notifications, and search pages. The controller uses Cache::remember() for the photo list and single-photo retrieval, and Cache::forget() after upload and deletion to reduce stale data.

Nginx Proxy Configuration: Nginx was updated with a /storage/ proxy rule. This is important because the frontend and API are exposed through the Nginx service on port 8080, while Laravel serves the actual uploaded file through its public storage path.

4.3 Troubleshooting Technical Issues

1. Internal Storage Framework Paths: Laravel required valid framework storage directories. Ensuring the storage directories exist and clearing cache resolved internal path errors.

2. Table Creation & Migrations: The photos table was persisted via migration execution, verified with migrate:status.

3. Redis Cache Configuration: Cache invalidation and memory retrieval were implemented in the controller and verified via Redis PING.

5. Conclusion

Project 8 successfully extends the existing web system with a simplified Photo Upload Service. The implementation creates the photos table, defines the Photo model, validates and stores image uploads, persists metadata in PostgreSQL, returns structured JSON responses, exposes public image URLs, simulates asynchronous processing, registers versioned API routes, and includes caching logic for metadata retrieval.

The screenshots prove the main implementation and testing requirements: migration execution, controller logic, storage link, API routes, successful upload with HTTP 201 Created, successful list and single-photo retrieval with HTTP 200 OK, validation error with HTTP 422, uploaded file existence, Redis availability, and Git commit. The module follows the architectural ideas from the lecture: media files should be stored outside the database, metadata should be cached, processing should be asynchronous in real systems, and scalable social platforms should separate media storage, APIs, cache, and delivery concerns.

6. References

1. Vistula University. Web Systems Design - Meeting 8 Project: Photo Upload Service laboratory instructions, 14.05.2026.
2. Vistula University. Web Systems Design - Meeting 8 Lecture: Instagram-like System architecture, 11.05.2026.
3. Laravel Documentation. Filesystem: <https://laravel.com/docs/filesystem>
4. Laravel Documentation. Validation: <https://laravel.com/docs/validation>
5. Laravel Documentation. HTTP Requests - Retrieving Uploaded Files: <https://laravel.com/docs/requests#retrieving-uploaded-files>
6. Laravel Documentation. File Uploads: <https://laravel.com/docs/filesystem#file-uploads>
7. Laravel Documentation. Eloquent ORM: <https://laravel.com/docs/eloquent>
8. Laravel Documentation. Cache: <https://laravel.com/docs/cache>
9. Laravel Documentation. Queues: <https://laravel.com/docs/queues>
10. Docker Documentation. Compose overview: <https://docs.docker.com/compose/>
11. PostgreSQL Documentation: <https://www.postgresql.org/docs/>
12. Redis Documentation: <https://redis.io/docs/latest/>
13. Nginx Documentation: <https://nginx.org/en/docs/>